
Module 04 — HTTP Clients and Resilience

Lesson Notes

Spring Boot Edu



2026-09-23

Contents

1. Learning Goals	2
2. Concepts	2
2.1 HTTP Clients in Spring	2
2.2 Failures in Distributed Systems	3
3. Step-by-Step Examples	3
3.1 RestClient	3
3.2 Turning HTTP Errors into Domain Exceptions	4
3.3 Shared Settings for Every Client	5
3.4 HTTP Interface Clients	5
3.5 For Comparison: WebClient	6
3.6 Retrying with @Retryable (Spring Framework 7)	7
3.7 Preventing Overload with @ConcurrencyLimit	9
3.8 Timeouts	9
4. Common Mistakes and Best Practices	10
5. Summary	11
6. Further Reading	11

1. Learning Goals

By the end of this module you will be able to:

- Call another service with `RestClient` and turn HTTP errors into domain exceptions
- Add shared headers and interceptors to every client
- Define an HTTP client with nothing but an interface (HTTP interface)
- See how a reactive call with `WebClient` differs
- Survive transient failures and overload with Spring Framework 7's `@Retryable` and `@ConcurrencyLimit`
- Configure timeouts
- Fake external services in tests with `WireMock`

Prerequisites: Modules 01-03 · **Estimated time:** 4 hours

2. Concepts

2.1 HTTP Clients in Spring

Client	Model	When?
<code>RestClient</code>	Synchronous, fluent API	The default choice. Scales together with virtual threads
HTTP interface (<code>@HttpExchange</code>)	An interface, Spring generates the implementation	When calling a service with many endpoints
<code>WebClient</code>	Reactive (Mono/Flux)	When the application is reactive anyway (Module 13)
<code>RestTemplate</code>	Old, synchronous	Do not use it in new code

Spring Boot hands out a ready `RestClient.Builder` bean: JSON converters, timeouts and `RestClientCustomizers` are already applied. It also picks the underlying HTTP library from the classpath. The order of preference is Apache `HttpClient`, Jetty, Reactor Netty, the JDK `HttpClient`, then the JDK `URLConnection`. `spring.http.clients.imperative.factory` pins the choice.

2.2 Failures in Distributed Systems

A remote service can slow down, return 503 for a while or not answer at all. Resilience patterns prepare you for these situations:

Pattern	What it does	In this module
Timeout	Prevents waiting forever	<code>spring.http.clients.read-timeout</code>
Retry	Waits and tries again on transient failures	<code>@Retryable</code>
Bulkhead / concurrency limit	Limits how many calls run at the same time	<code>@ConcurrencyLimit</code>
Fallback	Gives a meaningful substitute answer on failure	<code>try/catch + last known value</code>

! Important

Retry **transient** failures only (5xx, timeouts). A 404 or 400 does not get better by trying again. Also make sure the retried operation is idempotent.

3. Step-by-Step Examples

Start the application:

```
./mvnw -pl modules/04-http-clients-resilience/lesson -am spring-boot:run
```

The “remote” catalog service runs inside the lesson application as `FakeCatalogController`. Its price endpoint returns 503 twice per ISBN, and its cover endpoint takes 200 ms. The clients connect to it over real HTTP. In tests, WireMock takes its place.

3.1 RestClient

Goal: read JSON from another service and turn it into a record.

```
public CatalogRestClient(RestClient.Builder builder, CatalogProperties properties) {
    this.restClient = builder // Boot's builder: timeouts,
        ↪ customizers, JSON
        .baseUrl(properties.baseUrl())
        .build();
}
```

```
}

public BookInfo find(String isbn) {
    return restClient.get()
        .uri("/catalog/books/{isbn}", isbn)           // URI template: the value is
        ↪ encoded safely
        .retrieve()
        .onStatus(status -> status.value() == 404, (request, response) -> {
            throw new BookInfoNotFoundException(isbn);
        })
        .onStatus(HttpStatus::is5xxServerError, (request, response) -> {
            throw new CatalogUnavailableException("Catalog answered " +
                ↪ response.getStatusCode());
        })
        .body(BookInfo.class);                       // JSON → record
}
```

Expected output:

```
== 3.1 RestClient
BookInfo[isbn=9780134685991, title=Effective Java, authors=[Joshua Bloch], pageCount=412]
```

Its test: catalog/CatalogRestClientTest. WireMock fakes the remote service:

```
@Test
void readsABook() {
    cata-
    ↪ log.stubFor(get("/catalog/books/9780134685991").willReturn(okJson(EFFECTIVE_JAVA)));
    ↪ // fake response

    assertThat(client.find("9780134685991"))
        .isEqualTo(new BookInfo("9780134685991", "Effective Java",
            ↪ java.util.List.of("Joshua Bloch"), 412));
}
```

3.2 Turning HTTP Errors into Domain Exceptions

Goal: the calling code sees meaningful errors, not HTTP details.

The `onStatus` calls in section 3.1 do this: 404 → `BookInfoNotFoundException`, 5xx → `CatalogUnavailableException`. Without `onStatus`, `RestClient` would throw `HttpClientErrorException` / `HttpServerErrorException`.

Expected output:

```
== 3.2 Error handling
```

```
BookInfoNotFoundException: The catalog has no book with ISBN 9780000000000
```

3.3 Shared Settings for Every Client

Goal: add headers such as User-Agent and a request id to every client, in one place.

Boot applies every `RestClientCustomizer` bean to each `RestClient.Builder`:

```
@Component
public class CorrelationIdCustomizer implements RestClientCustomizer {

    @Override
    public void customize(RestClient.Builder builder) {
        builder.defaultHeader(HttpHeaders.USER_AGENT, "bookstore/1.0")
            .requestInterceptor((request, body, execution) -> {
                request.getHeaders().add("X-Request-Id",
↪      UUID.randomUUID().toString()); // trace a call across services
                return execution.execute(request, body);
            });
    }
}
```

Its test: `CatalogRestClientTest.everyRequestCarriesACorrelationIdAndUserAgent`.
WireMock verifies the headers of the incoming request.

3.4 HTTP Interface Clients

Goal: describe a service's endpoints with an interface only. Spring generates the implementation.

```
@HttpExchange("/catalog/books")
public interface CatalogApi {

    @GetExchange("/{isbn}")
    BookInfo find(@PathVariable String isbn);

    @GetExchange("/{isbn}/price")
    Price price(@PathVariable String isbn);

    @GetExchange("/{isbn}/cover")
    byte[] cover(@PathVariable String isbn);
}
```

Spring Framework 7's `@ImportHttpServices` turns interfaces into beans, organised as a **group**:

```
@Configuration(proxyBeanMethods = false)
@ImportHttpServices(group = "catalog", types = CatalogApi.class)
@EnableConfigurationProperties(CatalogProperties.class)
public class HttpClientsConfiguration {
}
```

The group's address and timeouts come from `application.yaml`:

```
http:
  clients:                                # defaults for every HTTP client Boot creates (Lesson
    ↪ 3.8)
    connect-timeout: 2s
    read-timeout: 5s
    imperative:
      factory: jdk                        # RestClient on the JDK HttpClient (Reactor Netty is
    ↪ only for WebClient here)
    serviceclient:
      catalog:                           # the group of @ImportHttpServices(group = "catalog")
    ↪ (Lesson 3.4)
      base-url: http://localhost:${server.port:8080}
      read-timeout: 3s                   # overrides the default for this group only
```

Expected output:

```
== 3.4 HTTP interface client
Spring in Action
```

Its test: `catalog/CatalogApiTest`

3.5 For Comparison: WebClient

Goal: see the reactive version of the same call.

```
@Component
public class ReactiveCatalogClient {

    private final WebClient webClient;

    public ReactiveCatalogClient(WebClient.Builder builder, CatalogProperties
    ↪ properties) {
        this.webClient = builder.baseUrl(properties.baseUrl()).build();
    }
}
```

```
}

public Mono<BookInfo> find(String isbn) {
    return webClient.get()
        .uri("/catalog/books/{isbn}", isbn)
        .retrieve()
        .bodyToMono(BookInfo.class);           // nothing happens until
        ↪ someone subscribes
}
}
```

A Mono is lazy: no request is sent until `block()` or `subscribe()` is called. In a servlet (Web MVC) application, `RestClient` on virtual threads is simpler and sufficient. `WebClient` is meant for reactive applications (Module 13).

Expected output:

```
== 3.5 WebClient
Mono → 412 pages
```

i Note

The `WebClient` starter brings `Reactor Netty`, and `Boot` would pick that library for `RestClient` too. In this module, `spring.http.clients.imperative.factory: jdk` pins `RestClient` to the JDK's `HttpClient`.

3.6 Retrying with @Retryable (Spring Framework 7)

Goal: retry transient failures with increasing waits, without any extra library.

First the feature is switched on:

```
@Configuration(proxyBeanMethods = false)
@EnableResilientMethods
public class ResilienceConfiguration {
}
```

The retried call:

```
@Component
public class PriceQuery {

    private final CatalogApi catalog;

    public PriceQuery(CatalogApi catalog) {
```

```
        this.catalog = catalog;
    }

    @Retryable(
        includes = HttpServerErrorException.class,    // only 5xx: a 404 will not get
        ↪ better by retrying
        maxRetries = 3,                               // 1 call + up to 3 retries
        delay = 100, multiplier = 2, jitter = 20)      // wait ~100 ms, ~200 ms, ~400 ms
    public Price fetch(String isbn) {
        return catalog.price(isbn);
    }
}
```

If every attempt fails, the caller falls back to the last known price:

```
public Price priceOrLastKnown(String isbn) {
    try {
        return currentPrice(isbn);
    } catch (RestClientException exception) {
        log.warn("Catalog unavailable after retries, using the last known price: {}",
        ↪ exception.getMessage());
        Price fallback = lastKnown.get(isbn);
        if (fallback == null) {
            throw exception;
        }
        return fallback;
    }
}
```

Expected output (the fake catalog answers 503 twice):

```
== 3.6 @Retryable (the fake catalog fails twice, then answers)
price 89.90 after 335 ms
```

Its test: resilience/PriceServiceTest. WireMock’s scenario feature scripts “two failures, then success”, and the test checks that exactly 3 requests arrived. It also checks that a 404 is not retried.

! Warning

@Retryable works through a proxy. If PriceService called this.fetch(...), there would be **no** retry (self-invocation, Module 01). That is why the retried method lives in a separate bean, PriceQuery.

3.7 Preventing Overload with @ConcurrencyLimit

Goal: respect a partner’s rule of “at most 2 requests at the same time”.

```
@ConcurrencyLimit(2)                                // the 3rd caller waits (policy
↳ BLOCK, the default)
public byte[] download(String isbn) {
    maxInFlight.accumulateAndGet(inFlight.incrementAndGet(), Math::max);
    try {
        return catalog.cover(isbn);
    } finally {
        inFlight.decrementAndGet();
        completed.incrementAndGet();
    }
}
```

The default policy is BLOCK, so extra callers wait in line. policy = REJECT refuses them immediately with an InvocationRejectedException (Exercise 3). With virtual threads there can be thousands of callers, which makes this limit especially important.

Expected output (6 downloads of 200 ms each):

```
== 3.7 @ConcurrencyLimit(2): 6 downloads of 200 ms each
max in flight 2, took 648 ms
```

Three waves × 200 ms ≈ 600 ms. Without the limit it would take about 200 ms.

Its test: resilience/CoverServiceTest

★ Tip

Uncomment org.springframework.resilience: debug in application.yaml to watch callers being held and released in the log.

3.8 Timeouts

Goal: stop an unresponsive service from blocking your application.

The defaults live under spring.http.clients.*. A group can get its own values under spring.http.serviceclient.<group>.* (the YAML in section 3.4). The test sets the read timeout to 300 ms and expects a response delayed by 2 seconds. Instead of waiting 2 seconds, the call fails quickly with a ResourceAccessException:

```
@TestPropertySource(properties = "spring.http.clients.read-timeout=300ms")
class TimeoutTest extends WireMockCatalogTest {
```

```
@Autowired
CatalogRestClient client;

@Test
void aSlowResponseTimesOut() {
    cata-
    ↪ log.stubFor(get("/catalog/books/9780134685991").willReturn(okJson(EFFECTIVE_JAVA).withFixedDelay(2

        long start = System.nanoTime();
        assertThatThrownBy(() ->
            ↪ client.find("9780134685991")).isInstanceOf(ResourceAccessException.class);
        org.assertj.core.api.Assertions.assertThat((System.nanoTime() - start) /
    ↪ 1_000_000).isLessThan(1_500);
    }
}
```

!! Caution

An HTTP call without a timeout holds your thread indefinitely when the remote service hangs (and, even with virtual threads, your connection and memory). Set a deliberate timeout for every client.

4. Common Mistakes and Best Practices

! Warning

Spring Test caches test contexts. If you start WireMock with a JUnit extension **per test class**, each class gets a new random port, while the cached context keeps using the old one, and you get “Connection refused”. The tests of this module start one WireMock server once per JVM (WireMockCatalogTest).

- **Do:** use the `RestClient.Builder` that Boot provides, not `RestClient.create()`. Otherwise you lose the settings and customizers.
- **Don't:** leak HTTP error classes (`HttpClientErrorException`) into the business layer. Turn them into domain exceptions with `onStatus`.
- **Do:** retry transient failures only, a limited number of times, with increasing waits (multiplier, jitter).
- **Don't:** retry the same call in both the client and the calling service. The attempts multiply.
- **Do:** fake external services in tests with a stub server such as WireMock, and verify the requests it received.

5. Summary

- `RestClient` is the default HTTP client. `onStatus` turns HTTP errors into domain exceptions.
- A `RestClientCustomizer` adds settings to every client from one place.
- With HTTP interfaces and `@ImportHttpServices` you call services without writing client code. The settings live under `spring.http.serviceclient.<group>`.
- In Spring Framework 7, `@Retryable` and `@ConcurrencyLimit` are built in and switched on with `@EnableResilientMethods`.
- Timeouts are set with `spring.http.clients.*`.
- `WireMock` fakes external services reliably.

Next step: [Exercises](#)

6. Further Reading

- [Spring Framework — REST Clients](#)
- [Spring Framework — Resilience Features](#)
- [Spring Boot — Calling REST Services](#)
- [WireMock Documentation](#)